

Lab0 CPPWarmUp 实验报告

PB21020665 张之正

一、实验目的

动态数组优化：验证基于模板的动态数组（DArray）通过预分配内存（m_nMax）减少频繁内存分配带来的性能提升。

多项式类对比：比较基于 `std::list` 和 `std::map` 的多项式类（PolynomialList 和 PolynomialMap）在不同规模数据下的性能差异。

时间复杂度验证：分析链表（list）和有序映射（map）在插入、合并、运算等操作中的效率差异。

二、实现思路

1、动态数组优化

策略：引入容量变量 `m_nMax`，每次扩容时容量倍增（如从 $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$ ），减少 `new/delete` 次数。

核心方法：`Reserve(int nCapacity)` 实现预分配，`PushBack` 和 `InsertAt` 依赖此方法优化内存操作。

2、多项式类的两种实现

PolynomialList：使用 `std::list<Term>` 存储项，按次数降序排列。合并同类项需遍历链表，时间复杂度为 $O(n)$ 。

PolynomialMap：使用 `std::map<int, double>`，键为次数，值为系数。自动按次数排序，合并同类项时间复杂度为 $O(\log n)$ 。

3、测试设计

动态数组：通过插入、删除、扩容等操作验证内存管理正确性和输出格式。

多项式类：生成随机多项式数据，测试构造函数、加减乘运算的时间消耗。

三、实现过程

1、动态数组（DArray）

关键代码：

```
void PushBack(const T& dValue) {  
    if (m_nSize >= m_nMax) Reserve(m_nMax == 0 ? 1 : m_nMax * 2);  
    m_pData[m_nSize++] = dValue;  
}
```

测试用例：插入、删除、拷贝构造、赋值运算符，验证输出结果和容量变化。

2、多项式类（PolynomialList 和 PolynomialMap）

PolynomialList 合并同类项：

```
void AddOneTerm(const Term& term) {  
    // 遍历链表查找相同次数项，合并或插入  
    auto it = m_Polynomial.begin();  
    while (it != m_Polynomial.end() && it->deg > term.deg) { ... }  
}
```

PolynomialMap 合并同类项：

```
m_Polynomial[deg] += cof; // 自动合并  
if (m_Polynomial[deg] == 0.0) m_Polynomial.erase(deg);
```

文件读取：解析 data/P3.txt，处理乱序输入项并排序。

四、实验结果

(一)、运行结果如下

1、

```
-main/USTC.CG_25-main/Homeworks/0_cpp_warmup/project/bin/1_BasicDArray.exe
Array elements: 2.1
Array elements: 2.1, 3, 3.1, 3.2
Array elements: 3, 3.1, 3.2
Array elements: 4.1, 3, 3.1, 3.2
Array elements: 4.1, 3, 3.1, 3.2
Array elements: 4.1, 3, 3.1, 3.2
Array elements: 4.1, 3, 3.1, 3.2
Array elements: 4.1, 3, 3.1, 3.2
Array elements: 21
Empty array
Array elements: 22, 0, 0, 0, 0
Array elements: 100, 97, 98, 99
```

2、

```
PS C:\Users\0129294\Desktop\学习\USTC.CG_25-main\USTC.CG_25-main\Homeworks\0_cpp_warmup\project\bin> . "C:\Users\0129294\Desktop\学习\USTC.CG_25-main\USTC.CG_25-main\Homeworks\0_cpp_warmup\project\bin\2_EfficientDArray.exe"
Array(1/1): 2.1
Array(4/4): 2.1, 3, 3.1, 3.2
Array(3/4): 3, 3.1, 3.2
Array(4/4): 4.1, 3, 3.1, 3.2
Array(4/4): 4.1, 3, 3.1, 3.2
Array(1/1): 21
Empty array
Array(5/5): 22, -6.27744e+66, -6.27744e+66, -6.27744e+66, -6.27744e+66
Array(4/4): 100, 97, 98, 99
PS C:\Users\0129294\Desktop\学习\USTC.CG_25-main\USTC.CG_25-main\Homeworks\0_cpp_warmup\project\bin>
```

3、

```
PS C:\Users\0129294\Desktop\学习\USTC.CG_25-main\USTC.CG_25-main\Homeworks\0_cpp_warmup\project\bin> . "C:\Users\0129294\Desktop\学习\USTC.CG_25-main\USTC.CG_25-main\Homeworks\0_cpp_warmup\project\bin\3_TemplateDArray.exe"
size= 1: 2.1
size= 4: 2.1 3 3.1 3.2
size= 3: 3 3.1 3.2
size= 4: 4.1 3 3.1 3.2
size= 4: 4.1 3 3.1 3.2
size= 4: 4.1 3 3.1 3.2
size= 4: 4.1 3 3.1 3.2
size= 1: 21
size= 0:
size= 5: 22 0 0 0 0
size= 4: d a b c
PS C:\Users\0129294\Desktop\学习\USTC.CG_25-main\USTC.CG_25-main\Homeworks\0_cpp_warmup\project\bin>
```

4、

```
PS C:\Users\0129294\Desktop\学习\USTC_CG_25-main\USTC_CG_25-main\Homeworks\0_cpp_warmup\project\bin> . "C:/Users/0129294/Desktop/学习/USTC_CG_25-main/USTC_CG_25-main/Homeworks/0_cpp_warmup/project/bin/4_list_Polynomial.exe"
-3x^4 + 2x^1
-3x^4 + 4x^1
-6x^4 + 6x^1
-2x^1
9x^8 + -18x^5 + 8x^2
PS C:\Users\0129294\Desktop\学习\USTC_CG_25-main\USTC_CG_25-main\Homeworks\0_cpp_warmup\project\bin> 
```

5、

```
PS C:\Users\0129294\Desktop\学习\USTC_CG_25-main\USTC_CG_25-main\Homeworks\0_cpp_warmup\project\bin> . "C:/Users/0129294/Desktop/学习/USTC_CG_25-main/USTC_CG_25-main/Homeworks/0_cpp_warmup/project/bin/5_map_Polynomial.exe"
Test List:
280x^9352 + 240x^8578 + 2632x^8379 + 2431x^7605 + 3584x^7073 + 2352x^6714 + 1645x^6632 + 3252x^6299 + 2276x^5940 + 3932x^5326 + 3914x^4967 + 2304x^4020 + 4840x^3661 + 2184x^3302
56x^5063 + 53x^4289 + 82x^3316 + 100x^2010 + 94x^1651
56x^5063 + 43x^4289 + -12x^3316 + -28x^2010 + 10x^1651
Test Constructor time: 4

Test Map:
2184x^3302 + 4840x^3661 + 2304x^4020 + 3914x^4967 + 3932x^5326 + 2276x^5940 + 3252x^6299 + 1645x^6632 + 2352x^6714 + 3584x^7073 + 2431x^7605 + 2632x^8379 + 240x^8578 + 280x^9352
94x^1651 + 100x^2010 + 82x^3316 + 53x^4289 + 56x^5063
10x^1651 + -28x^2010 + -12x^3316 + 43x^4289 + 56x^5063
Test Constructor time: 5

Test List:
Test Constructor time: 2812

Test Map:
Test Constructor time: 10

Test List:
Test Constructor time: 10651

Test Map:
Test Constructor time: 22
```

（二）、动态数组测试

输出示例：

Array(1/1): 2.1

Array(4/4): 2.1, 3, 3.1, 3.2

Array(3/4): 3, 3.1, 3.2

分析：容量从 1 倍增到 4，删除后容量保持不变，内存管理高效。

（三）、多项式类性能对比

小规模数据（5 项）：

PolynomialList 构造时间：4ms

PolynomialMap 构造时间：5ms

中规模数据（100 项）：

PolynomialList 构造时间：2812ms

PolynomialMap 构造时间：10ms

大规模数据（150 项）：

PolynomialList 构造时间：10651ms

PolynomialMap 构造时间：22ms

五、结果分析

1、动态数组优化效果

优点：容量倍增策略将插入操作的平均时间复杂度从 $O(n^2)$ 降至 $O(n)$ ，减少内存分配次数。

问题：删除元素后未扩容，可能导致内存浪费（如从 4 容量删除到 3 项）。

2、多项式类性能对比

PolynomialList 缺点：

插入需遍历链表找到合适位置，时间复杂度 $O(n)$ 。

合并同类项需显式遍历，效率低。

PolynomialMap 优点：

自动按次数排序，插入和查找时间复杂度 $O(\log n)$ 。

合并同类项通过 `map[key] += value` 直接完成，无需遍历。

3、时间复杂度验证

构造时间对比：

数据规模 List 时间 (ms) Map 时间 (ms)

5 4 5

100 2812 10

150 10651 22

结论：map 实现在大规模数据下性能显著优于 list。

六、改进方向

1、动态数组：

实现扩容策略（如容量降至 1/4 时扩容一半）。

2、多项式类：

为 PolynomialList 优化插入逻辑（如二分查找插入位置）。

在 PolynomialMap 中预分配内存以减少哈希冲突。

七、总结

通过本实验验证了动态数组内存预分配的有效性，并对比了链表与映射在多项式运算中的性能差异。结果表明：

动态数组：通过容量倍增策略显著优化插入性能。

多项式类：`std::map` 在合并同类项和运算中具有明显优势，适合处理大规模稀疏多项式。